LAB 10

KIỂM THỬ PHẦN MỀM

TÀI LIỆU THỰC HÀNH

LAB 10 - API TESTING VỚI REST ASSURED + KIỂM THỬ TÍCH HỢP UI–API

Kiểm thử REST API | Kết hợp API + UI | Chương trình 7 bài tập

Nội dung	Chi tiết
Chủ đề	REST Assured, HTTP Methods, JSON Schema, Authorization, API+UI Integration
Kỹ năng đạt được	Kiểm thử toàn bộ vòng đời API CRUD, phát hiện lỗi sớm hơn UI test 10x
Công cụ	REST Assured 5.x, Jackson, TestNG, Allure, Maven
Thời gian	120 phút | Điểm số: 10 điểm
Điều kiện tiên quyết	Hiểu HTTP cơ bản (request/response, status code). Đã học Lab 9 (BaseTest)
Website thực hành	reqres.in (API public miễn phí), jsonplaceholder.typicode.com, saucedemo.com

Mục tiêu bài lab

Sau khi hoàn thành Lab 10, sinh viên có thể:

- Giải thích tại sao API test quan trọng hơn và nhanh hơn UI test
- Viết test kiểm thử đầy đủ GET, POST, PUT, DELETE với REST Assured
- Kiểm thử phản hồi lỗi (4xx, 5xx) và xử lý exception của API
- Thực hiện JSON Schema Validation để bảo vệ cấu trúc response
- Kiểm thử authorization: endpoint có/không có token
- Kết hợp API setup với UI test để tăng tốc và giảm flakiness
- Xây dựng ApiBaseTest tái sử dụng cho toàn dự án

I. TÓM TẮT LÝ THUYẾT

10.1 Tại sao phải kiểm thử API? Tháp kiểm thử (Test Pyramid)

- link tài liệu tham khảo:

- https://viblo.asia/p/gioi-thieu-ve-kiem-thu-api-aWj537Me56m
- https://viblo.asia/p/ket-hop-kiem-thu-ui-va-api-chien-luoc-qa-thong-nhat-cho-cac-doi-ngu-hien-dai-0gdJzzgjJz5
- https://viblo.asia/p/api-testing-voi-rest-assured-so-sanh-postman-vs-rest-assured-naQZRkGjlvx
- https://anhtester.com/blog/blog/so-luoc-ve-api-testing
- https://www.testing.vn/rest-api-test-automation/
- https://codelearn.io/sharing/hoc-api-automation-testing-voi-rest-assured

Hầu hết lỗi phần mềm không phát sinh ở giao diện - chúng xuất phát từ tầng xử lý nghiệp vụ và API. Kiểm thử chỉ qua UI giống như chỉ nhìn vào kết quả cuối mà không kiểm tra quá trình tính toán.

❖ **Test Pyramid - Triết lý kiểm thử hiện đại**

ĐỈNH THÁP - UI/E2E Test (5-10%): Chậm (~30s/test), flakey, chi phí bảo trì cao.

→ Chỉ test các luồng nghiệp vụ quan trọng nhất (thanh toán, đăng ký...).

GIỮA THÁP - API/Integration Test (20-30%): Trung bình (~100ms/test), ổn định.

→ Kiểm tra business logic, contract giữa các service, xử lý lỗi.

ĐÁY THÁP - Unit Test (60-70%): Nhanh nhất (~5ms/test), không cần server.

→ Developer viết, kiểm tra từng hàm/method độc lập.

Sai lầm phổ biến của team mới: viết quá nhiều UI test, bộ test chạy 4–8 tiếng.

- ➢ Hậu quả: không ai chạy test trước khi merge code → regression không được phát hiện.
- ✓ **Nguyên tắc vàng: Nếu có thể kiểm thử bằng API → ĐỪNG viết UI test cho tính năng đó.**

Tiêu chí so sánh	API Test	UI/Selenium Test
Tốc độ thực thi	~50–200ms mỗi test	~3000–10000ms mỗi test
Độ ổn định (Stability)	95%+ ổn định, ít flakey	60–80% ổn định (phụ thuộc network, render)
Chi phí bảo trì	Thấp (API ít đổi hơn UI)	Cao (locator thay đổi thường xuyên)
Phát hiện lỗi	Ngay khi developer push code	Sau khi có giao diện hoàn chỉnh
Kỹ năng cần thiết	Hiểu HTTP, JSON	Hiểu DOM, CSS Selector, Wait
Công cụ	REST Assured, Postman, Newman	Selenium, Playwright, Cypress

10.2 HTTP và REST-Nền tảng cần nắm vững

HTTP Method	Mục đích	Request Body	Status Code thành công	Ví dụ URL
GET	Lấy dữ liệu	Không có	200 OK	/api/users/2
POST	Tạo mới resource	Có (JSON body)	201 Created	/api/users
PUT	Cập nhật toàn bộ	Có (JSON body)	200 OK	/api/users/2
PATCH	Cập nhật một phần	Có (JSON một phần)	200 OK	/api/users/2
DELETE	Xóa resource	Không có	204 No Content	/api/users/2

Status Code	Nhóm	Ý nghĩa	Khi nào xuất hiện trong test
200 OK	2xx Success	Request thành công, có data trả về	GET, PUT thành công
201 Created	2xx Success	Tạo resource thành công	POST thành công
204 No Content	2xx Success	Thành công, không có data trả về	DELETE thành công
400 Bad Request	4xx Client Error	Request sai định dạng hoặc thiếu field	POST thiếu required field
401 Unauthorized	4xx Client Error	Chưa xác thực (thiếu/sai token)	Gọi API cần auth mà không có token

403 Forbidden	4xx Client Error	Không có quyền dù đã xác thực	User thường cố xóa dữ liệu admin
404 Not Found	4xx Client Error	Resource không tồn tại	GET /api/users/999999
500 Internal Server Error	5xx Server Error	Lỗi phía server	Bug trong backend code

10.3 REST Assured - Cú pháp và cách sử dụng

Thêm dependency vào pom.xml:

```
<dependencies>
  <!-- REST Assured: Thư viện chính để test API -->
  <dependency>
    <groupId>io.rest-assured</groupId>
    <artifactId>rest-assured</artifactId>
    <version>5.4.0</version>
    <scope>test</scope>
  </dependency>

  <!-- JSON Schema Validator: Kiểm tra cấu trúc JSON response -->
  <dependency>
    <groupId>io.rest-assured</groupId>
    <artifactId>json-schema-validator</artifactId>
    <version>5.4.0</version>
    <scope>test</scope>
  </dependency>
  <!-- Jackson Databind: Serialize/Deserialize JSON <-> Java object
-->

  <dependency>
    <groupId>com.fasterxml.jackson.core</groupId>
    <artifactId>jackson-databind</artifactId>
    <version>2.17.0</version>
  </dependency>
</dependencies>
```

Cú pháp Given–When–Then - Đọc như ngôn ngữ tự nhiên:

```
// Import tĩnh — viết ở đầu file để code gọn
import static io.restassured.RestAssured.*;
import static org.hamcrest.Matchers.*;
import static io.restassured.module.jsv.JsonSchemaValidator.*;

// Cấu hình chung cho toàn bộ test class — đặt trong @BeforeClass
@BeforeClass
public void setup() {
```

```
    RestAssured.baseURI = "https://reqres.in";
    RestAssured.basePath = "/api";
    // Bật logging: in request + response khi test FAIL — rất hữu
ích để debug

RestAssured.enableLoggingOfRequestAndResponseIfValidationFails();
}

// ===== GET — Lấy danh sách =====
given()
    .queryParam("page", 2)          // ?page=2
    .header("Accept", "application/json")
.when()
    .get("/users")                  // GET
https://reqres.in/api/users?page=2
.then()
    .statusCode(200)               // Kiểm tra HTTP status
    .body("page", equalTo(2))       // JSON path: $.page == 2
    .body("data", not(empty()))     // $.data không rỗng
    .body("data[0].id", notNullValue())  // Phần tử đầu tiên có id
    .body("data[0].email", containsString("@"));  // Email hợp lệ

// ===== GET — Lấy một bản ghi =====
given()
.when()
    .get("/users/2")
.then()
    .statusCode(200)
    .body("data.id",         equalTo(2))
    .body("data.email",      equalTo("janet.weaver@reqres.in"))
    .body("data.first_name", equalTo("Janet"));

// ===== GET — Resource không tồn tại =====
given()
.when()
    .get("/users/9999")
.then()
    .statusCode(404)
    .body("$", anEmptyMap());  // Body phải là {} (object rỗng)

// ===== POST — Tạo mới =====
// Cách 1: Dùng HashMap
Map<String, Object> body = new HashMap<>();
body.put("name", "Nguyen Van A");
body.put("job",  "QA Engineer");
```

```
Response response = given()
    .contentType("application/json")
    .body(body)                     // Tự động serialize sang JSON
.when()
    .post("/users")
.then()
    .statusCode(201)
    .body("name",       equalTo("Nguyen Van A"))
    .body("id",         notNullValue())   // ID được tạo tự động bởi
server
    .body("createdAt", notNullValue()   // Timestamp tạo mới
    .extract().response();

String newUserId = response.jsonPath().getString("id");
System.out.println("ID user vừa tạo: " + newUserId);

// ===== PUT — Cập nhật =====
given()
    .contentType("application/json")
    .body("{\"name\":\"Tran Thi B\",\"job\":\"Senior QA\"}")
.when()
    .put("/users/2")
.then()
    .statusCode(200)
    .body("job",        equalTo("Senior QA"))
    .body("updatedAt", notNullValue());

// ===== DELETE =====
given()
.when()
    .delete("/users/2")
.then()
    .statusCode(204);  // No Content — body rỗng
```

10.4 JSON Schema Validation - Bảo vệ cấu trúc response

Trong hệ thống nhiều team, một backend developer có thể đổi tên field (vd: first_name → firstName) mà không thông báo. JSON Schema Validation phát hiện lỗi này ngay lập tức:

```
// src/test/resources/schemas/user-schema.json
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "type": "object",
  "required": ["data", "support"],
  "properties": {
```

```
    "data": {
      "type": "object",
      "required": ["id", "email", "first_name",
"last_name", "avatar"],
      "properties": {
        "id":         { "type": "integer", "minimum": 1
},
        "email":      { "type": "string",  "format":
"email" },
        "first_name": { "type": "string",  "minLength": 1
},
        "last_name":  { "type": "string",  "minLength": 1
},
        "avatar":     { "type": "string",  "format":
"uri" }
      },
      "additionalProperties": false
    },
    "support": {
      "type": "object",
      "required": ["url", "text"],
      "properties": {
        "url":  { "type": "string", "format": "uri" },
        "text": { "type": "string" }
      }
    }
  }
}
// Sử dụng trong test:
@Test(description = "Response phải khớp với JSON Schema
đã định nghĩa")
public void testUserSchemaValidation() {
    given()
    .when()
        .get("/users/2")
    .then()
        .statusCode(200)
        // Fail nếu backend đổi 'first_name' →
'firstName'
```

```
        .body(matchesJsonSchemaInClasspath("schemas/user-
schema.json"));
}
// Lỗi sẽ là: Schema Validation failed. Tên field
'first_name' không tìm thấy.
```

10.5 RequestSpecification - Tránh lặp code cấu hình

Trong dự án thực, mỗi API call đều có chung header, auth token, base URL. Dùng RequestSpecification và ResponseSpecification để cấu hình một lần, dùng nhiều lần:

```
// ApiBaseTest.java — Lớp cha cho tất cả API test class
public class ApiBaseTest {
    protected RequestSpecification requestSpec;
    protected ResponseSpecification responseSpec;
    @BeforeClass
    public void setupApiSpec() {
        // RequestSpecification: header + base URL chung
        requestSpec = new RequestSpecBuilder()
            .setBaseUri("https://reqres.in")
            .setBasePath("/api")
            .setContentType(ContentType.JSON)
            .addHeader("Accept", "application/json")
            // Log toàn bộ request + response khi test
fail
            .addFilter(new RequestLoggingFilter())
            .addFilter(new ResponseLoggingFilter())
            .build();
        // ResponseSpecification: assertion chung
        responseSpec = new ResponseSpecBuilder()
            .expectContentType(ContentType.JSON)
            .expectResponseTime(lessThan(3000L))  // SLA:
tất cả API phải trả về < 3s
            .build();
    }
    // Method tiện ích để lấy auth token
    protected String getAuthToken() {
        return given(requestSpec)
.body("{\"email\":\"eve.holt@reqres.in\",\"password\":\"c
ityslicka\"}")
```

```
        .when()
            .post("/login")
        .then()
            .statusCode(200)
            .extract().jsonPath().getString("token");
    }
}
// Sử dụng trong test class:
public class UserApiTest extends ApiBaseTest {
    @Test
    public void testGetUser() {
        given(requestSpec)          // Dùng spec chung
        .when()
            .get("/users/2")
        .then()
            .spec(responseSpec)     // Assertion chung
(SLA response time)
            .statusCode(200);
    }
}
```

10.6 Kiểm thử Authorization

Hầu hết API thực tế đều cần xác thực (authentication). Kiểm thử authorization đảm bảo API bảo vệ đúng endpoint:

Kịch bản kiểm thử	Mong đợi		Lý do quan trọng
Gọi API không cần auth → không có token	200 OK		Endpoint public phải hoạt động bình thường
Gọi API cần auth → có token hợp lệ	200 OK		Flow chuẩn khi user đã đăng nhập
Gọi API cần auth → không có token	401 Unauthorized		Bảo vệ dữ liệu người dùng

Gọi API cần auth → token sai/hết hạn	401 Unauthorized		Phòng chống tấn công token cũ
Gọi API với token đúng nhưng sai quyền	403 Forbidden		User thường không xóa được data admin
Gọi với token đúng định dạng nhưng bịa	401 Unauthorized		Phòng chống forge token

```
// AuthorizationTest.java
public class AuthorizationTest extends ApiBaseTest {
    @Test(description = "API login trả về token hợp lệ")
    public void testLoginReturnsToken() {
        String token = given(requestSpec)
.body("{\"email\":\"eve.holt@reqres.in\",\"password\":\"c
ityslicka\"}")
        .when()
            .post("/login")
        .then()
            .statusCode(200)
            .body("token", notNullValue())
            .body("token", not(emptyString()))
            .extract().jsonPath().getString("token");
        Assert.assertFalse(token.isEmpty(), "Token không
được rỗng");
    }

    @Test(description = "Đăng nhập thiếu password → 400
Bad Request")
    public void testLoginMissingPassword() {
        given(requestSpec)
            .body("{\"email\":\"eve.holt@reqres.in\"}")
        .when()
            .post("/login")
        .then()
            .statusCode(400)
            .body("error", equalTo("Missing password"));
    }
```

```
    @Test(description = "Đăng nhập thiếu email → 400 Bad
Request")
    public void testLoginMissingEmail() {
        given(requestSpec)
            .body("{\"password\":\"secret\"}")
        .when()
            .post("/login")
        .then()
            .statusCode(400)
            .body("error", equalTo("Missing email or
username"));
    }
}
```

10.7 Kết hợp API Test và UI Test - Kỹ thuật giảm thời gian test

❖ **Tại sao kết hợp API + UI?**

Vấn đề: Một test E2E checkout cần user đã đăng nhập và có hàng trong giỏ.

Nếu setup tất cả qua UI: đăng ký user (15s) + đăng nhập (5s) + thêm hàng (3s) = 23 giây chỉ để setup.

Với 50 test cần setup như vậy: 50 × 23s = 19 phút TRƯỚC khi bắt đầu test thực sự.

Giải pháp - API Precondition Setup:

BƯỚC 1 (API): POST /login → lấy session token (< 200ms)

BƯỚC 2 (API): Xác nhận user tồn tại và có quyền (< 100ms)

BƯỚC 3 (UI): Inject session cookie/localStorage vào browser

BƯỚC 4 (UI): Navigate thẳng đến trang cần test (bỏ qua trang login)

Kết quả: Setup từ 23 giây xuống còn < 1 giây - nhanh hơn 23 lần.

Thực tế: Nhiều công ty lớn (Netflix, Google) dùng kỹ thuật này trong suite test hàng triệu test case.

```
// Ví dụ: Kết hợp API login + UI test (áp dụng với saucedemo)
public class ApiUiIntegrationTest extends BaseTest {
    @BeforeMethod
```

```
    public void setupViaSauceDemoLogin() {
        // SauceDemo dùng đơn giản: set localStorage sau khi load
trang
        // Thực tế dùng cách: gọi API lấy cookie rồi inject vào
browser
        JavascriptExecutor js = (JavascriptExecutor) getDriver();

        // Mô phỏng: inject session state qua JS thay vì điền form
login
        // (Thực tế với hệ thống có API: inject JWT token hoặc
session cookie)
        js.executeScript(
            "window.localStorage.setItem('session-username',
'standard_user')"
        );

        // Điều hướng thẳng đến inventory — không qua trang login

getDriver().get("https://www.saucedemo.com/inventory.html");
    }

    @Test(description = "UI test checkout bắt đầu thẳng từ trang
sản phẩm")
    public void testCheckoutWithApiPrecondition() {
        InventoryPage inventory = new InventoryPage(getDriver());
        Assert.assertTrue(inventory.isLoaded(), "Trang inventory
phải load được");

        inventory.addFirstItemToCart();
        Assert.assertEquals(inventory.getCartItemCount(), 1);

        CartPage cart = inventory.goToCart();
        Assert.assertEquals(cart.getItemCount(), 1, "Giỏ hàng phải
có 1 sản phẩm");
    }
}
```

10.8 Kiểm thử Response Time - SLA Performance Test

Mọi API trong dự án thực đều có Service Level Agreement (SLA) về thời gian phản hồi. Vi phạm SLA ảnh hưởng trải nghiệm người dùng. REST Assured có thể kiểm tra điều này:

```
// PerformanceAssertionTest.java
```

```
public class PerformanceAssertionTest extends ApiBaseTest
{
    // SLA: API danh sách user phải trả về trong 2 giây
    @Test(description = "GET /users phải phản hồi trong
2000ms")
    public void testListUsersResponseTime() {
        given(requestSpec)
        .when()
            .get("/users")
        .then()
            .statusCode(200)
            .time(lessThan(2000L));  // Milliseconds
    }
    // Kiểm tra + lấy thời gian để log
    @Test(description = "Đo thời gian phản hồi và kiểm
tra ngưỡng")
    public void testMeasureAndAssertResponseTime() {
        long startTime = System.currentTimeMillis();
        given(requestSpec)
        .when()
            .get("/users/1")
        .then()
            .statusCode(200);
        long elapsed = System.currentTimeMillis() -
startTime;
        System.out.println("[Perf] GET /users/1 mất: " +
elapsed + "ms");
        Assert.assertTrue(elapsed < 3000,
            "Response time " + elapsed + "ms vượt SLA
3000ms");
    }
    // Kiểm tra nhiều API cùng lúc
    @DataProvider(name = "apiEndpoints")
    public Object[][] apiEndpoints() {
        return new Object[][] {
            { "/users",   200, 2000L },
            { "/users/1", 200, 2000L },
            { "/users/2", 200, 2000L },
        };
    }
```

```
    @Test(dataProvider = "apiEndpoints", description =
"SLA cho nhiều endpoint")
    public void testSLAForAllEndpoints(String endpoint,
int status, long maxMs) {
        given(requestSpec).when().get(endpoint).then()
            .statusCode(status)
            .time(lessThan(maxMs));
    }
}
```

II. BÀI TẬP THỰC HÀNH

Bài 1 THIẾT LẬP API BASETEST VÀ KIỂM THỬ GET CƠ BẢN (1 điểm)

Tạo lớp nền tảng ApiBaseTest và viết test kiểm thử GET request trên https://reqres.in:

- Triển khai ApiBaseTest với RequestSpecification (baseURI, contentType, logging filter) và ResponseSpecification (contentType, response time < 3000ms)
- Test 1: GET /api/users?page=1 → Kiểm tra status 200, page=1, total_pages > 0, data.size() >= 1
- Test 2: GET /api/users?page=2 → Kiểm tra page=2, mỗi user trong data[] có id, email, first_name, last_name, avatar
- Test 3: GET /api/users/3 → Kiểm tra id=3, email đúng định dạng @reqres.in, first_name không rỗng
- Test 4: GET /api/users/9999 → Kiểm tra status 404, body là object rỗng {}
- Mỗi test phải dùng requestSpec và responseSpec từ ApiBaseTest - không viết lại cấu hình

Bài 2 KIỂM THỬ POST, PUT, DELETE - CRUD ĐẦY ĐỦ (2 điểm)

Kiểm thử toàn bộ vòng đời của resource User trên reqres.in - mô phỏng luồng thực tế: tạo → đọc → cập nhật → xóa.

Bài test	Endpoint	Assertion bắt buộc	Điểm
POST tạo user	POST /api/users	201, body.name khớp input, body.id không null, body.createdAt không null	0.5đ
PUT cập nhật user	PUT /api/users/2	200, body.job khớp input, body.updatedAt không null, updatedAt khác createdAt nếu có	0.5đ
PATCH cập nhật một phần	PATCH /api/users/2	200, chỉ field được gửi thay đổi, updatedAt mới hơn	0.25đ
DELETE xóa user	DELETE /api/users/2	204, response body rỗng hoàn toàn	0.25đ
POST → GET xác nhận	POST tạo, lấy id, GET để xác nhận	201 → 200, data trả về khớp với data đã tạo	0.5đ

Yêu cầu kỹ thuật:

- Tạo POJO class CreateUserRequest (name, job) và UserResponse (name, job, id, createdAt) với Jackson annotations
- Dùng POJO thay vì String JSON trong request body: given().body(new CreateUserRequest("Tên", "Job"))
- Lưu id từ POST response, dùng lại trong test GET xác nhận (chứng tỏ biết cách chain các API call)

Bài 3 JSON SCHEMA VALIDATION (1 điểm)

Tạo JSON Schema cho các endpoint chính và viết test kiểm thử cấu trúc response không bị thay đổi:

- Tạo file schemas/user-list-schema.json kiểm tra response GET /api/users: phải có page (integer), total_pages (integer), data (array), mỗi item trong data phải có id, email, first_name, last_name, avatar
- Tạo file schemas/user-schema.json kiểm tra response GET /api/users/2: cấu trúc nested object data và support
- Tạo file schemas/create-user-schema.json kiểm tra response POST /api/users: phải có name (string), job (string), id (string), createdAt (string)
- Viết test cho cả 3 schema, đặt additionalProperties: false trong schema → test sẽ FAIL nếu backend thêm field mới không khai báo
- Demo: Thêm field không có trong schema vào expected body → test phải FAIL với thông báo rõ ràng về field bị thiếu/thừa

Bài 4 KIỂM THỬ AUTHORIZATION VÀ ERROR HANDLING (1.5 điểm)

Kiểm thử các kịch bản liên quan đến xác thực và xử lý lỗi:

Phần A - Authorization (1.0 điểm):

- Test login thành công: POST /api/login với email và password hợp lệ → 200, có token không rỗng
- Test login thiếu password → 400, error = "Missing password"
- Test login thiếu email → 400, error = "Missing email or username"

- Test register thành công: POST /api/register → 200, có id và token
- Test register thiếu password → 400, error = "Missing password"

Phần B - Data-Driven cho Error Handling (0.5 điểm):

- Tạo @DataProvider cho 5 kịch bản login với input khác nhau (email hợp lệ/không hợp lệ, password có/không)
- Mỗi test case: {email, password, expectedStatus, expectedErrorMessage}

```
@DataProvider(name = "loginScenarios")
public Object[][] loginScenarios() {
    return new Object[][] {
        // email,                    password,
expectedStatus, expectedError
        {"eve.holt@reqres.in",      "cityslicka", 200,
null},
        {"eve.holt@reqres.in",      "",           400,
"Missing password"},
        {"",                        "cityslicka", 400,
"Missing email or username"},
        {"notexist@reqres.in",      "wrongpass",  400,
"user not found"},
        {"invalid-email",           "pass123",    400,
"user not found"},
    };
}
@Test(dataProvider = "loginScenarios")
public void testLoginScenarios(String email, String
password,
                               int expectedStatus, String
expectedError) {
    Map<String,String> body = new HashMap<>();
    body.put("email", email);
    if (!password.isEmpty()) body.put("password",
password);
    ValidatableResponse response =
given(requestSpec).body(body)

.when().post("/login").then().statusCode(expectedStatus);
```

```
    if (expectedError != null) {
        response.body("error",
containsString(expectedError));
    }
}
```

Bài 5 PERFORMANCE ASSERTION VÀ SLA MONITORING (1 điểm)

Xây dựng bộ kiểm thử giám sát SLA (Service Level Agreement) cho các API chính:

Endpoint	SLA tối đa	Assertion bổ sung
GET /api/users	2000ms	data.size() >= 1
GET /api/users/2	1500ms	data.id = 2
POST /api/users	3000ms	201 Created, có id
POST /api/login	2000ms	200, có token
DELETE /api/users/2	1000ms	204 No Content

- Viết test dùng @DataProvider cho bảng trên - 1 method test, 5 data row
- Thêm Allure @Step annotation: "Gọi {method} {endpoint} - SLA: {maxMs}ms"
- Sau khi chạy, chụp console log cho thấy thời gian thực tế của mỗi API call
- Tạo một test chạy cùng API 10 lần liên tiếp, tính average/min/max response time và log ra - mô phỏng monitoring đơn giản

Bài 6 TÍCH HỢP API + UI: SETUP QUA API, VERIFY QUA UI (2 điểm)

Kịch bản tích hợp thực tế: Dùng API để chuẩn bị trạng thái, dùng UI để xác minh kết quả cuối cùng.

Phần A - API Precondition → UI Verification (1.0 điểm):

- Viết @BeforeMethod: Gọi POST /api/login trên reqres.in để lấy token. Log token ra console để xác nhận
- Nếu API login fail (status != 200) → dùng dependsOnMethods để SKIP test UI (không chạy test UI khi điều kiện tiên quyết thất bại)
- Trong test UI: Đăng nhập vào saucedemo.com bằng cách nhập form (không inject), nhưng chỉ chạy khi API precondition pass
- Verify: URL sau đăng nhập chứa 'inventory', title trang là 'Swag Labs'

Phần B - Luồng tích hợp đầy đủ (1.0 điểm):

- Gọi GET /api/users để xác nhận API reqres.in đang sống (status 200). Lưu kết quả vào biến isApiAlive
- Nếu isApiAlive = false → test UI sẽ được đánh dấu là SKIP (dùng throw new SkipException())
- Thực hiện luồng UI: saucedemo đăng nhập → thêm 2 sản phẩm → kiểm tra badge = 2 → vào giỏ → xác nhận 2 sản phẩm
- Viết bình luận trong code giải thích từng bước: đây là bước API check, đây là bước UI action, đây là assertion

Bài 7 DỰ ÁN TỔNG HỢP: KIỂM THỬ API JSONPLACEHOLDER (1.5 điểm)

Áp dụng tất cả kỹ năng đã học vào API mới: https://jsonplaceholder.typicode.com - API giả lập blog với Posts, Comments, Users.

Tài liệu API cần kiểm thử

GET /posts - Lấy 100 bài đăng

GET /posts/1 - Lấy bài đăng id=1

GET /posts/1/comments - Lấy comments của bài 1

POST /posts - Tạo bài đăng mới (title, body, userId)

PUT /posts/1 - Cập nhật toàn bộ bài đăng

DELETE /posts/1 - Xóa bài đăng

GET /users - Lấy danh sách users (10 user)

GET /users/1 - User có id, name, username, email, address (nested object)

Yêu cầu tối thiểu:

1. Tạo PostSchema.json và UserSchema.json kiểm tra cấu trúc response. User schema phải validate nested object address (city, zipcode)
2. Viết PostApiTest kiểm thử đầy đủ CRUD cho /posts: GET danh sách, GET một bản ghi, POST tạo mới, PUT cập nhật, DELETE

3. Viết CommentApiTest: GET /posts/1/comments → 5 comment, mỗi comment có postId=1, id, name, email, body
4. Viết UserApiTest: GET /users → 10 user; GET /users/1 → validate schema bao gồm nested address object
5. Tất cả test đều có SLA assertion: response time < 3000ms
6. Chạy toàn bộ suite với Allure report. Chụp màn hình report hiển thị tất cả test PASS

➢ **Tiêu chí chấm điểm toàn bài**

- ✓ ApiBaseTest được tái sử dụng đúng cách - không có duplicate cấu hình
- ✓ POJO class cho request/response body - không dùng String JSON thuần túy
- ✓ JSON Schema validation cho ít nhất 3 endpoint khác nhau
- ✓ Data-Driven test với @DataProvider cho ít nhất 1 kịch bản
- ✓ Bài 6: API precondition thực sự ảnh hưởng đến việc có chạy UI test hay không
- ✓ TestNG HTML Report: tất cả test pass, tên test mô tả rõ scenario đang kiểm thử

HẾT

Good Luck!

Học để nuôi bồ & nuôi ta!!!